RoboTrot: Robust Quadruped Locomotion on AMD Radeon GPUs

AMD AI DevMaster Hackathon — Track 3: Physical AI
Team: [YOUR TEAM NAME]

1. Overview

This project demonstrates robust quadruped locomotion in simulation, combined with an AMD Radeon GPU / ROCm-accelerated perception module for real-time terrain awareness. The submission targets Track 3 (Physical AI), focusing on robotic locomotion, simulation-based training/evaluation, and ROCm adoption for on-device inference.

2. Problem Statement

Most hackathon-scale legged robot demos only show locomotion on flat, ideal terrain. Real-world deployment requires robustness to unexpected disturbances (pushes, collisions) and varying terrain friction (pavement vs. gravel vs. wet/slippery surfaces). This project explicitly tests and measures robustness under both conditions, and adds a perception component that could enable future closed-loop terrain adaptation.

3. Approach

3.1 Locomotion Controller

A Central Pattern Generator (CPG) produces a trot gait: diagonal leg pairs (front-right + rear-left, front-left + rear-right) move in phase, 180° out of phase with the opposite pair. Joint targets from the CPG are tracked using PD control (Kp=60, Kd=2) on all 12 actuated joints of the Unitree Go1 model. A classical control approach was chosen deliberately over training an RL policy from scratch, to eliminate training-convergence risk within the hackathon's short timeline, and to focus engineering effort on the robustness testing and GPU-accelerated perception, which are the project's novel contributions.

3.2 Robustness Testing

Each simulated episode runs the same gait controller under two ground-friction regimes (high friction, representing pavement; low friction, representing gravel or wet surfaces) and injects four randomized horizontal pushes to the robot's torso at random points during the episode. We log: whether the robot falls (torso height collapse), maximum body tilt, and net distance traveled.

3.3 GPU-Accelerated Perception (AMD Radeon / ROCm)

A lightweight convolutional neural network classifies onboard camera image patches into terrain categories (pavement, gravel, grass, slippery). This model is designed to run on-device in real time, so its inference latency was benchmarked directly on AMD Radeon GPU hardware via PyTorch's ROCm backend, and compared against CPU-only inference. This benchmark demonstrates the throughput

needed to support future real-time gait adaptation, and is the project's direct evidence of AMD Radeon GPU / ROCm adoption.

4. System Architecture

Component	Description	Location
Robot model	Unitree Go1 quadruped (MuJoCo Menagerie)	assets/unitree_go1/
Gait controller	CPG-based trot gait generator	sim/gait_controller.py
Simulation runner	MuJoCo physics sim, PD control, push/terrain randomization, video export	sim/run_sim.py
Perception model	Tiny CNN terrain classifier + CPU/ROCm benchmark	perception/terrain_classifier.py
Environment setup	Installs MuJoCo, PyTorch+ROCm, dependencies	setup_rocm.sh

5. Judging-Criteria Mapping (Track 3)

Criterion	Points	How this project addresses it
Robot capability performance	30	Trot gait sustains locomotion across two terrain frictions and survives randomized pushes v
AMD Radeon GPU / ROCm adoption	20	Perception CNN benchmarked on ROCm GPU vs. CPU for real-time inference latency and
Innovation & originality	20	Combines a reliable classical gait controller with GPU-accelerated terrain perception and e
Real-world application value	20	Terrain-aware locomotion and disturbance rejection are genuine deployment requirements
Contributions to open-source projects	10	Built on and credits MuJoCo Menagerie (Apache-2.0); code structured for reuse/extension

6. Results

Results are generated by running `sim/run_sim.py --compare`, which produces `results.json` with per-terrain robustness metrics (fall/no-fall, max tilt, distance traveled), and by running `perception/terrain_classifier.py --benchmark`, which prints CPU vs. ROCm GPU inference latency (ms/batch) and throughput (images/sec).

[Insert your actual results.json numbers and ROCm benchmark output here after running on your AMD Radeon GPU instance, before final submission.]

7. How to Run

```
bash setup_rocm.sh
cd sim && python3 run_sim.py --duration 15 --compare --video ../docs/demo.mp4
cd ../perception && python3 terrain_classifier.py --benchmark
```

8. Future Work

- Close the loop: feed live terrain classification into the gait controller to adapt stance width and speed in real time.
- Replace the CPG with a learned residual policy layered on top of it for further performance gains without losing reliability.
- Add domain randomization (mass, motor delay, friction) for sim-to-real transfer toward physical Radeon-powered robot hardware.

9. Attribution

Robot model: MuJoCo Menagerie — Unitree Go1 (Apache-2.0 License), https://github.com/google-deepmind/mujoco_menagerie. Simulation: MuJoCo (https://mujoco.org/). Perception model implemented and benchmarked with PyTorch + ROCm.